Dynamic Programming Study Guide

What is Dynamic Programming?

Dynamic Programming (DP) is a problem-solving technique that breaks down complex problems into simpler subproblems. It stores the results of subproblems to avoid redundant calculations, making algorithms more efficient.

Key Characteristics:

- **Optimal Substructure**: The optimal solution contains optimal solutions to subproblems
- **Overlapping Subproblems**: The same subproblems are solved multiple times
- **Memoization**: Storing results to avoid recalculation

Dynamic Programming Approaches

1. Top-Down Approach (Memoization)

Uses recursion with a cache to store previously computed results.

2. Bottom-Up Approach (Tabulation)

Builds up the solution iteratively from smaller subproblems.

Practice Session on Dynamic Programming

Fibonacci Type Questions

1. Basic Fibonacci Sequence

Problem: Find the nth Fibonacci number where F(0) = 0, F(1) = 1, and F(n) = F(n-1) + F(n-2).

Naive Recursive Solution:

```java
public class Fibonacci {
    public static int fibonacciNaive(int n) {
        if (n <= 1) {
            return n;
        }
        return fibonacciNaive(n - 1) + fibonacciNaive(n - 2);
    }
}
```

Memoization Solution:

```java
import java.util.HashMap;
import java.util.Map;

public class Fibonacci {
    private Map<Integer, Integer> memo = new HashMap<>();

    public int fibonacciMemo(int n) {
        if (n <= 1) {
            return n;
        }

        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        int result = fibonacciMemo(n - 1) + fibonacciMemo(n - 2);
        memo.put(n, result);
        return result;
    }
}
```

Bottom-Up Solution:

```java
public class Fibonacci {
    public static int fibonacciDP(int n) {
        if (n <= 1) {
            return n;
        }

        int[] dp = new int[n + 1];
        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
}
```

Space-Optimized Solution:

```java
public class Fibonacci {
    public static int fibonacciOptimized(int n) {
        if (n <= 1) {
            return n;
        }

        int prev2 = 0, prev1 = 1;
        int current = 0;

        for (int i = 2; i <= n; i++) {
            current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }

        return current;
    }
}
```

2. Climbing Stairs Problem

Problem: You're climbing a staircase with n steps. You can climb either 1 or 2 steps at a time. How many distinct ways can you climb to the top?

```java
public class ClimbingStairs {
    public static int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }

        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;

        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
}
```

3. House Robber Problem

Problem: You are a robber planning to rob houses along a street. Adjacent houses have security systems connected. Find the maximum amount you can rob without alerting the police.

```java
public class HouseRobber {
    public static int rob(int[] nums) {
        if (nums.length == 0) return 0;
        if (nums.length == 1) return nums[0];

        int[] dp = new int[nums.length];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);

        for (int i = 2; i < nums.length; i++) {
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
        }

        return dp[nums.length - 1];
    }
}
```

Grid-Based Questions

1. Unique Paths Problem

Problem: A robot is located at the top-left corner of an m × n grid. It can only move either down or right. How many possible unique paths are there to reach the bottom-right corner?

```java
public class UniquePaths {
    public static int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];

        // Initialize first row and column
        for (int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }
        for (int j = 0; j < n; j++) {
            dp[0][j] = 1;
        }

        // Fill the dp table
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

2. Minimum Path Sum

Problem: Given a grid filled with non-negative numbers, find a path from top-left to bottom-right that minimizes the sum of all numbers along its path. You can only move right or down.

```java
public class MinimumPathSum {
    public static int minPathSum(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;

        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];

        // Initialize first row
        for (int j = 1; j < n; j++) {
            dp[0][j] = dp[0][j - 1] + grid[0][j];
        }

        // Initialize first column
        for (int i = 1; i < m; i++) {
            dp[i][0] = dp[i - 1][0] + grid[i][0];
        }

        // Fill the dp table
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = Math.min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j];
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

3. Unique Paths with Obstacles

Problem: A robot is located at the top-left corner of an m × n grid. Some cells are blocked (marked as 1). Find the number of unique paths to reach bottom-right corner.

```java
public class UniquePathsWithObstacles {
    public static int uniquePathsWithObstacles(int[][] obstacleGrid) {
        int m = obstacleGrid.length;
        int n = obstacleGrid[0].length;

        if (obstacleGrid[0][0] == 1 || obstacleGrid[m - 1][n - 1] == 1) {
            return 0;
        }

        int[][] dp = new int[m][n];
        dp[0][0] = 1;

        // Initialize first row
        for (int j = 1; j < n; j++) {
            dp[0][j] = (obstacleGrid[0][j] == 0 && dp[0][j - 1] == 1) ? 1 : 0;
        }

        // Initialize first column
        for (int i = 1; i < m; i++) {
            dp[i][0] = (obstacleGrid[i][0] == 0 && dp[i - 1][0] == 1) ? 1 : 0;
        }

        // Fill the dp table
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (obstacleGrid[i][j] == 0) {
                    dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
                } else {
                    dp[i][j] = 0;
                }
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

4. Maximum Square in Binary Matrix

Problem: Given a 2D binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

```java
public class MaximalSquare {
    public static int maximalSquare(char[][] matrix) {
        if (matrix == null || matrix.length == 0) {
            return 0;
        }

        int rows = matrix.length;
        int cols = matrix[0].length;
        int[][] dp = new int[rows + 1][cols + 1];
        int maxSide = 0;

        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= cols; j++) {
                if (matrix[i - 1][j - 1] == '1') {
                    dp[i][j] = Math.min(Math.min(dp[i - 1][j], dp[i][j - 1]),
                                        dp[i - 1][j - 1]) + 1;
                    maxSide = Math.max(maxSide, dp[i][j]);
                }
            }
        }

        return maxSide * maxSide;
    }
}
```

Key DP Patterns

Pattern 1: Linear DP

Problems that can be solved by making decisions at each step based on previous steps.

- Fibonacci numbers
- Climbing stairs
- House robber

Pattern 2: Grid DP

Problems involving 2D grids where we need to find optimal paths or counts.

- Unique paths
- Minimum path sum

- Grid with obstacles

Pattern 3: String DP

Problems involving string manipulation and comparison.

- Longest common subsequence
- Edit distance

Pattern 4: Knapsack DP

Problems involving optimal selection with constraints.

- 0/1 knapsack
- Coin change

Time and Space Complexity Analysis

Problem	Naive Approach	DP Approach	Space Complexity
Fibonacci	O(2^n)	O(n)	O(n) or O(1)
Climbing Stairs	O(2^n)	O(n)	O(n) or O(1)
Unique Paths	O(2^(m+n))	O(m×n)	O(m×n)
Min Path Sum	O(2^(m+n))	O(m×n)	O(m×n) or O(n)

Quiz Questions

1. What is the time complexity of the naive recursive solution for finding the nth Fibonacci number? a) O(n) b) $O(n^2)$ c) O(2^n) d) O(log n)

Answer: c) O(2^n)

2. In the bottom-up approach for dynamic programming, we: a) Start from the final result and work backwards b) Use recursion with memoization c) Build up the solution from smaller subproblems iteratively d) Always use more space than top-down approach

Answer: c) Build up the solution from smaller subproblems iteratively

3. For the Unique Paths problem in an m×n grid, what is the value of dp[0][j] for all j? a) 0 b) 1 c) j d) undefined

Answer: b) 1

4. Which of the following problems does NOT follow the Fibonacci-type pattern? a) Climbing stairs (1 or 2 steps) b) House robber problem c) Minimum path sum in grid d) Tribonacci sequence

Answer: c) Minimum path sum in grid

5. What is the space complexity of the space-optimized Fibonacci solution? a) O(n) b) $O(n^2)$ c) O(1) d) O(log n)

Answer: c) O(1)

Practice Coding Questions

1. Coin Change Problem Given an array of coin denominations and a target amount, find the minimum number of coins needed to make up that amount. If it's not possible, return -1.

Example: coins = [1, 3, 4], amount = 6
Output: 2 (3 + 3)

2. Longest Increasing Subsequence Given an integer array, find the length of the longest strictly increasing subsequence.

Example: nums = [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4 (subsequence: [2, 3, 7, 101])

3. Paint House There are n houses in a row, each can be painted in red, blue, or green. The cost of painting each house with each color is different. No two adjacent houses can have the same color. Find the minimum cost to paint all houses.

Example: costs = [[17,2,17],[16,16,5],[14,3,19]]
Output: 10 (house 0: blue, house 1: green, house 2: blue)

4. Triangle Minimum Path Sum Given a triangle array, find the minimum path sum from top to bottom. You can move to adjacent numbers on the row below.

Example: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]
Output: 11 (path: 2 + 3 + 5 + 1 = 11)

5. Maximum Product Subarray Given an integer array, find the contiguous subarray that has the largest product and return the product.

Example: nums = [2,3,-2,4]
Output: 6 (subarray: [2,3])